

Git

Source Code Management

Git

Git is Management tool to store in source in Git...

Git is a modern and widely used **distributed version control** system in the world.

Git via the command line as well as with [GitHub](#).

What is Git

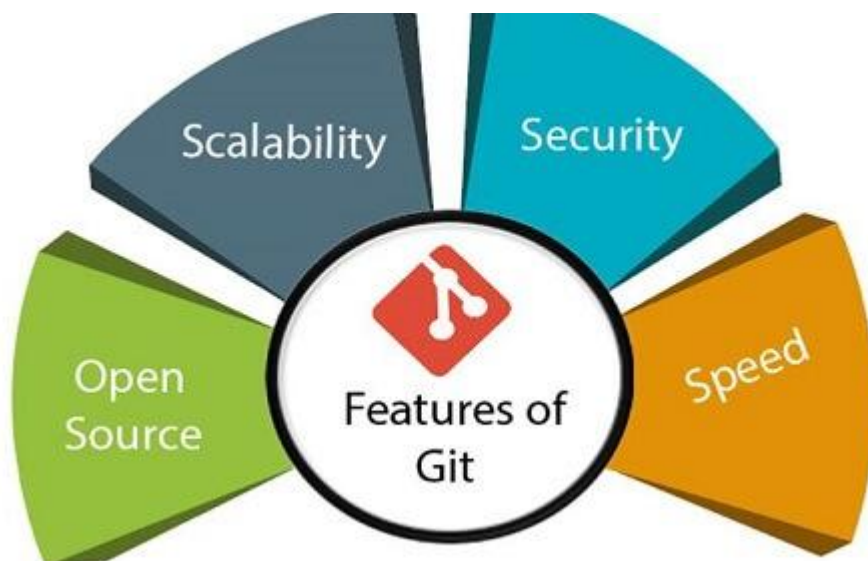
Git is an **open-source distributed version control system**. It is designed to handle minor to major projects with high speed and efficiency. It is developed to co-ordinate the work among the developers

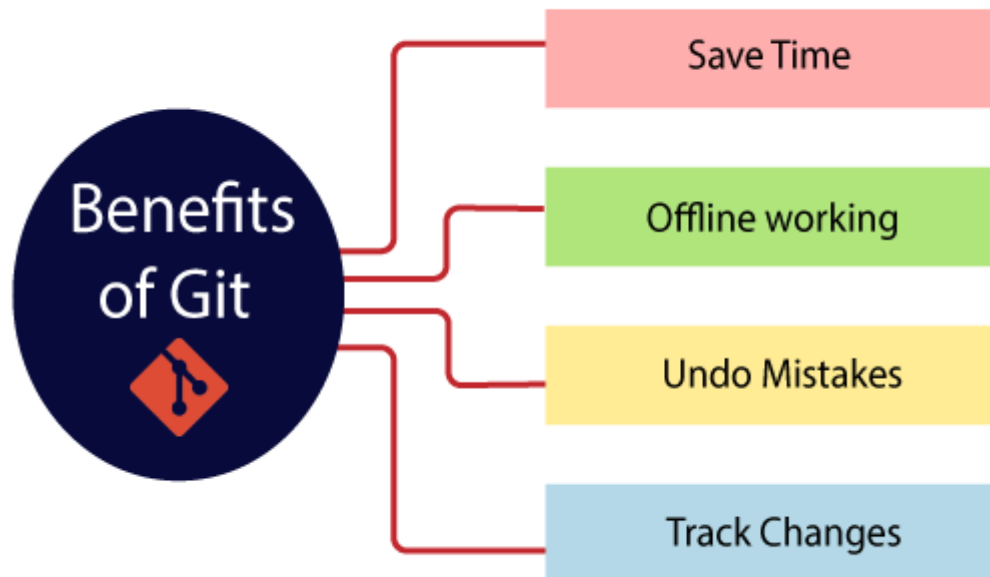
Git is foundation of many services like **GitHub** and **GitLab**, but we can use Git without using any other Git services. Git can be used **privately** and **publicly**.

Git was created by **Linus Torvalds** in **2005** to develop Linux Kernel. It is also used as an important distributed version-control tool for **the DevOps**.

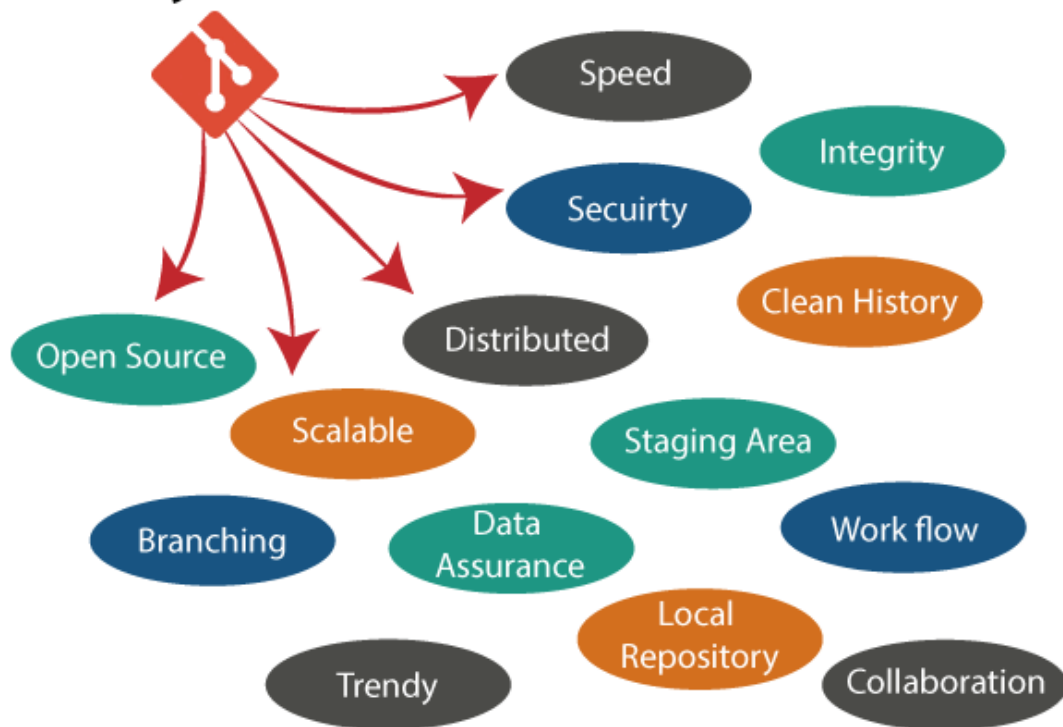
Git is easy to learn, and has fast performance

Features of Git





Why Git?



t

How to Install Git on Windows

To download the Git installer, visit the Git's official site and go to download page. The link for the download page is <https://git-scm.com/downloads>

Install Git:

Select **yes** to continue.

After the selecting **yes** the installation begins, and the screen

Click on **next** to continue.

Default components are automatically selected in this step. You can also choose your required part

Click on **next** to continue.

The default Git command-line options are selected automatically. You can choose your preferred choice.

Click **next** to continue.

The default transport backend options are selected in this step.

Click **next** to continue.

Select your required line ending option and

click **next** to continue.

Select preferred terminal emulator

clicks on the **next** to continue.

This is the last step that provides some extra features like system caching, credential management and symbolic link. Select the required features

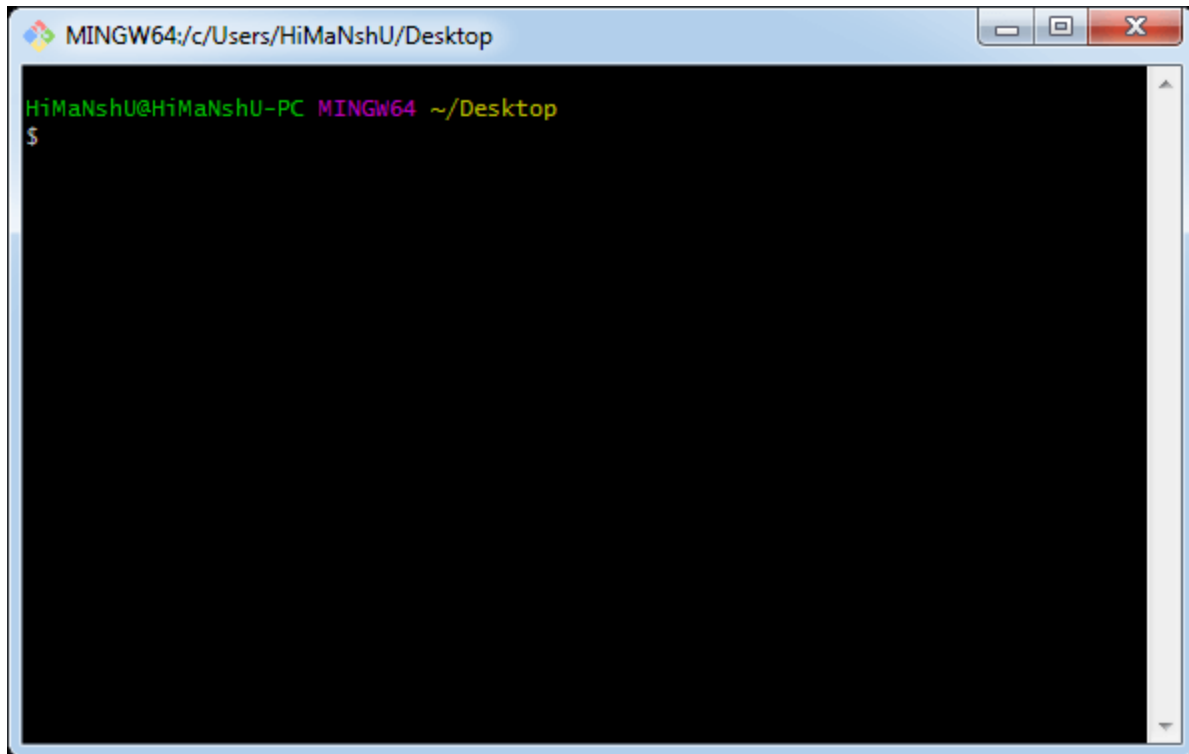
click on the **next** option.

The files are being extracted in this step.

t

The Git installation is completed.

The **Git Gui** looks like as



Git is a distributed version control system (VCS) used for tracking changes in source code during software development. It was created by Linus Torvalds in 2005 to manage the development of the Linux kernel. Git is designed to be fast, efficient, and flexible, making it a popular choice for managing version history, collaborating on code, and coordinating the work of multiple developers on a project.

Here's a breakdown of some key concepts and terms related to Git:

t

Repository (Repo): A repository is a directory or storage space where your project's files and version history are stored. It contains all the files, commits, branches, and other data related to your project.

Commit: A commit is a snapshot of the changes you've made to your code at a specific point in time. Each commit is accompanied by a message that describes the changes you've made.

Branch: A branch is a separate line of development within a repository. Branches allow multiple developers to work on different features or fixes simultaneously without interfering with each other's work. The main branch is typically called "master" or "main."

Merge: Merging is the process of integrating changes from one branch into another. It's used to combine the work done in different branches and create a unified history.

Pull Request (PR): In collaborative environments like GitHub or GitLab, a pull request is a request to merge changes from one branch into another. It allows for code review and discussion before changes are merged.

Clone: Cloning is the process of creating a copy of a repository on your local machine. This copy includes all the version history and branches, allowing you to work on the project locally.

Remote: A remote is a version of the repository that is hosted on a server, like GitHub, GitLab, or Bitbucket. It provides a centralized location for collaborating and sharing code with others.

Push: Pushing refers to sending your committed changes from your local repository to a remote repository. This makes your changes available to others and updates the remote repository's history.

t

Pull: Pulling is the process of fetching changes from a remote repository and integrating them into your local repository. It's used to stay up-to-date with the latest changes made by others.

Conflict: A conflict occurs when Git cannot automatically merge changes from different branches due to conflicting modifications in the same lines of code. Resolving conflicts requires manual intervention.

Version Control: Version control is the practice of tracking and managing changes to files over time. It allows developers to collaborate effectively, revert to previous states, and maintain a clear history of changes.

Common Git Workflows

Gitflow Workflow

- Uses **main** + **develop** branches.
- Feature branches → merged into **develop**.
- **Release branches** → created from develop → merged into main after testing.
- **Hotfix branches** → from main for urgent fixes.
- Good for projects with scheduled releases.

Key Commands in Git Workflow

- `git clone <repo>` → clone repo
- `git branch feature-x` → create branch
- `git checkout feature-x` → switch branch
- `git add .` & `git commit -m "msg"` → stage & commit
- `git push origin feature-x` → push branch
- Create **Pull Request/Merge Request** → get code reviewed
- `git merge feature-x` → merge into target branch

Branching Strategies in Git :

1. Master/Main Branch

- Represents **production-ready code**.
- Should always be stable; commits correspond to official releases.
- Example answer:
“The master branch reflects the latest production code. Only thoroughly tested and approved releases are merged here, often tagged for versioning.”

2. Develop Branch

- Integration branch for **ongoing development**.
- Feature branches are merged here after initial testing.
- Example answer:
“Develop is the integration branch. Completed features and bug fixes are merged here for further testing before a release.”

3. Feature Branches

- Created from **develop** for specific tasks or features.
- Isolated workspace; merged back into develop when done.
- Example answer:
“Feature branches allow developers to work on isolated functionalities without affecting the main codebase. Once complete and tested, they merge back into develop.”

4. Release Branches

- Created from **develop** when preparing for a release.
- Used for **final testing and bug fixes**; no new features.
- Merged into both **master and develop** after release.
- Example answer:
“Release branches are used for stabilization and last-minute fixes. Once the release is ready, it’s merged into master and develop, and tagged for versioning.”

5. Child Branches (Experimental / Hotfix / Isolated Work)

- Branches off main or develop for **isolated development**.
- Used for experiments, bug fixes, or isolated features.
- Example answer:
“Child branches create an isolated workspace. Developers can experiment or fix bugs without impacting the stable codebase. Code is usually reviewed via pull requests before merging.”

♦ Common Git Commands

1. Configuration

```
git config --global user.name "Your Name"  
git config --global user.email you@example.com
```

t

2. Repository Setup

```
git init          # Initialize a new Git repository
git clone <repo_url> # Clone an existing repository
```

3. Basic File Operations

```
git status      # Show repo status
git add .       # Add all changes
git commit -m "msg" # Commit changes
git log         # View commit history
```

4. Branching & Merging

```
git branch      # List branches
git branch feature-x # Create branch
git checkout feature-x # Switch branch
git merge feature-x   # Merge branch into current
```

5. Remote Operations

```
git remote -v      # Show remotes
git push origin branch # Push changes
git pull origin branch # Fetch & merge from remote
git fetch origin    # Download without merge
```

6. Undo / Reset

```
git reset --hard HEAD~1 # Undo last commit
git revert <commit_id>   # Revert a commit
git checkout -- <file>   # Discard file changes
```

7. Stashing

```
git stash      # Save changes temporarily
git stash pop   # Apply stashed changes
```

8. Tagging

```
git tag v1.0      # Create tag
git push origin v1.0 # Push tag
```

What a PR Review

- A developer submits a PR with changes (features, bug fixes, or improvements).
- Reviewers (team members or leads) go through the code to check for:
 - Correctness: Does it work as intended?
 - Code style: Follows project conventions.
 - Security: No vulnerabilities introduced.
 - Performance: Efficient and scalable.
 - Documentation & Tests: Adequate comments, docstrings, and unit/integration tests.

2. Key Steps in a PR Review

1. **Understand the change**
 - Read PR description carefully.
 - Check related tickets or issues.
2. **Check the code**
 - Look at logic, algorithms, and structure.
 - Spot potential bugs or edge cases.
3. **Check tests**
 - Are there unit tests? Integration tests?
 - Do they cover major scenarios?
4. **Comment and suggest**
 - Leave constructive feedback or ask clarifying questions.
5. **Approve or request changes**
 - Approve if everything is good.
 - Request changes if there are critical issues.